

AI Assignment 4

Travelling Salesman Problem (TSP) Using Greedy Technique and Simulated Annealing

Sabbir Ahmmed
Student ID: 2205040

September 7, 2026

Contents

1	Assignment Overview	2
2	Problem Description	2
3	Input Format	2
4	Implementation	3
4.1	Greedy Algorithm (Nearest Neighbor)	3
4.2	Simulated Annealing	3
4.3	Key Implementation Details	3
5	Experimental Setup	4
6	Experiment 1: Greedy vs. Simulated Annealing	4
7	Experiment 2: Varying Initial Temperature (T_0)	5
8	Experiment 3: Varying Cooling Rate (α)	6
9	Experiment 4: Random vs. Greedy Initialization	7
10	Complexity Analysis	8
10.1	Greedy Technique	8
10.2	Simulated Annealing	8
11	Comparative Summary	9
12	Conclusion	9
13	Submission Contents	10

1 Assignment Overview

The Travelling Salesman Problem (TSP) is one of the most well-known NP-hard optimization problems in computer science. A salesman must visit a number of cities exactly once and return to the starting city, minimizing the total travelling cost.

This assignment implements two approaches:

1. **Greedy Technique (Nearest Neighbor)** — to quickly construct a feasible solution.
2. **Simulated Annealing** — a metaheuristic optimization technique designed to improve solutions and escape local optima.

The goal is to implement, test, compare, and analyze both techniques across multiple TSP instances of varying sizes.

2 Problem Description

Given N cities and a travelling cost between every pair of cities, the task is to find a tour that:

1. Starts from a specified starting city (City 0).
2. Visits every city exactly once.
3. Returns to the starting city.
4. Minimizes the total travelling cost.

For a tour $0 \rightarrow c_1 \rightarrow c_2 \rightarrow \dots \rightarrow c_{N-1} \rightarrow 0$, the total cost is:

$$\text{Cost} = \sum_{i=0}^{N-1} \text{cost}(c_i, c_{(i+1) \bmod N})$$

3 Input Format

The input is a text file with the following format:

```
N
c[0][0] c[0][1] c[0][2] ... c[0][N-1]
c[1][0] c[1][1] c[1][2] ... c[1][N-1]
...
c[N-1][0] c[N-1][1] ... c[N-1][N-1]
```

Assumptions:

- $2 \leq N \leq 500$
- $\text{cost}(i, i) = 0$
- All other costs are positive.
- The matrix may be symmetric or asymmetric.
- City numbering starts from 0.

4 Implementation

4.1 Greedy Algorithm (Nearest Neighbor)

The greedy heuristic builds a tour by always choosing the nearest unvisited city:

1. Start at City 0. Mark it as visited.
2. Among all unvisited cities, select the one with minimum travelling cost from the current city.
3. Move to the selected city, mark it as visited.
4. Repeat until all cities have been visited.
5. Return to City 0.

Time Complexity: $O(N^2)$ — At each of the N steps, we scan all N cities to find the nearest unvisited one.

4.2 Simulated Annealing

Simulated Annealing is inspired by the physical process of annealing in metallurgy. A solution is iteratively improved by:

1. **Initial Solution:** Either a random permutation or the greedy tour.
2. **Neighbor Generation:** Using 2-opt segment reversal: pick positions $1 \leq i < j < N$, reverse the segment $[i, j]$.
3. **Acceptance Rule:**
 - If $\Delta \leq 0$ (improvement): always accept.
 - If $\Delta > 0$ (worsening): accept with probability $P = e^{-\Delta/T}$.
4. **Cooling Schedule:** Geometric cooling $T_{k+1} = \alpha \cdot T_k$.
5. **Termination:** When $T < T_{\min}$ or maximum iterations reached.

Time Complexity: $O(I \cdot N)$ where I is the total number of iterations. Each iteration generates a neighbor in $O(N)$ (segment reversal) and recalculates the tour cost in $O(N)$.

4.3 Key Implementation Details

The codebase is organized into the following modular components:

Table 1: Implementation Functions

Function	Purpose
readInput()	Read cost matrix from file
calculateTourCost()	Compute total tour cost including return edge
greedyTSP()	Nearest Neighbor heuristic
generateRandomTour()	Random permutation with City 0 fixed
generateNeighbor()	2-opt segment reversal
simulatedAnnealing()	Full SA algorithm with metrics
printResults()	Formatted output
isValidTour()	Tour validation
generateEuclideanInstance()	Random Euclidean cost matrix generator

5 Experimental Setup

Five TSP instances of increasing size were generated using random Euclidean coordinates:

Table 2: Test Instances

Instance	Cities	Type	Seed
Test1_10	10	Euclidean	1244
Test2_20	20	Euclidean	1254
Test3_50	50	Euclidean	1284
Test4_100	100	Euclidean	1334
Test5_200	200	Euclidean	1434

Default SA parameters: $T_0 = 1000$, $\alpha = 0.995$, $T_{\min} = 0.001$, iterations per temperature = 100, maximum iterations = 100,000.

6 Experiment 1: Greedy vs. Simulated Annealing

For each instance, the greedy algorithm was run once, and SA was run 5 times with different seeds. Results:

Table 3: Experiment 1 Results

Instance	Cities	Greedy	SA Best	SA Avg	Improve%	Time(s)
Test1_10	10	2462	2376	2376	3.49%	0.0098
Test2_20	20	4916	4170	4176	15.17%	0.0123
Test3_50	50	7387	5887	5985	20.31%	0.0235
Test4_100	100	10379	8315	8556	19.89%	0.0342
Test5_200	200	13521	13349	13624	1.27%	0.0592

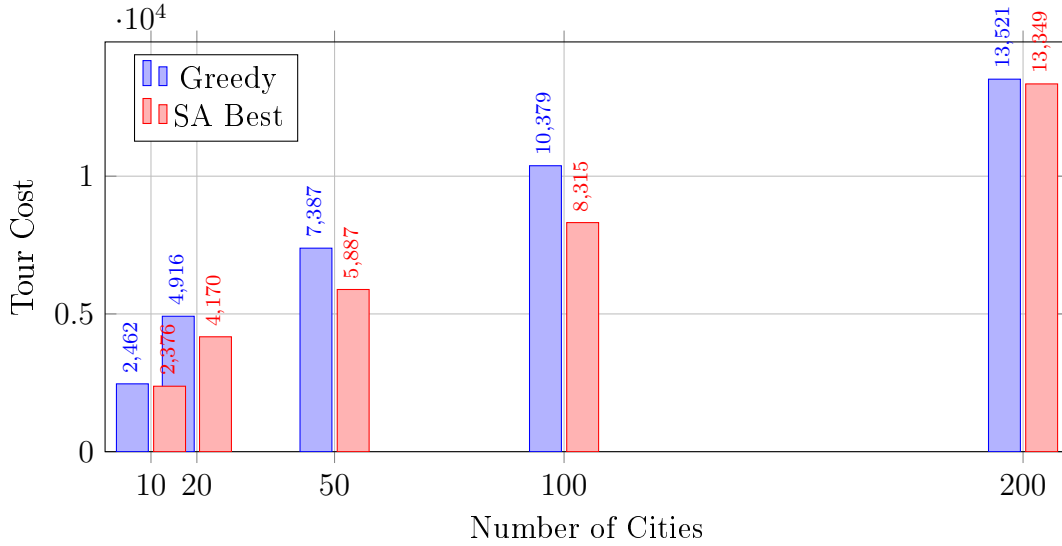


Figure 1: Greedy vs. SA Best Cost across instances

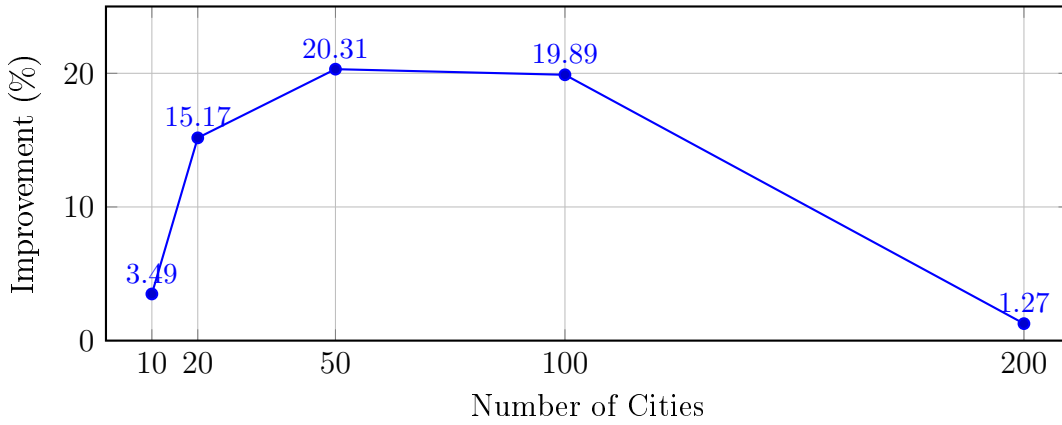


Figure 2: SA improvement over Greedy (%)

Discussion: SA consistently outperforms the greedy solution, with improvements ranging from 1.27% to 20.31%. The largest improvement is observed at $N = 50$ (20.31%). For the 200-city instance, the improvement is modest (1.27%), likely because the fixed iteration budget (100,000) becomes insufficient for larger instances. The greedy algorithm runs in microseconds while SA takes tens of milliseconds, but the solution quality improvement justifies the extra computation.

7 Experiment 2: Varying Initial Temperature (T_0)

Using the 50-city instance with $T_0 \in \{100, 1000, 5000\}$:

Table 4: Experiment 2 Results

T_0	Best Cost	Accepted Moves	Worse Accepted	Time(s)
100	5977	3565	647	0.0205
1000	5940	25976	11911	0.0197
5000	6508	54705	26249	0.0201

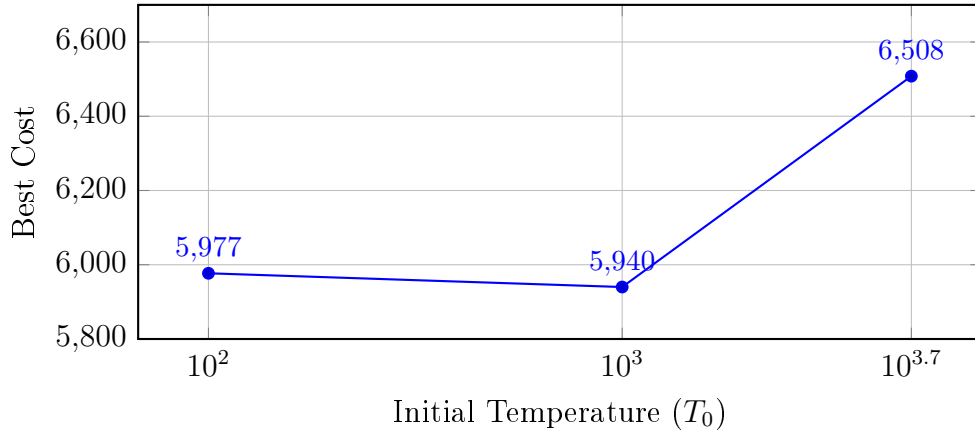


Figure 3: Best cost vs. initial temperature

Discussion:

- **Low T_0 (100):** Accepts only 647 worse moves. The algorithm behaves like a greedy local search — converges quickly but risks getting trapped in a local optimum (cost = 5977).
- **Moderate T_0 (1000):** Accepts 11,911 worse moves, achieving the best cost of 5940. This value balances exploration and exploitation within the iteration budget.
- **High T_0 (5000):** Accepts 26,249 worse moves — too much exploration early on wastes iterations before the temperature drops enough for the search to converge. The final cost (6508) is worse than even the low- T_0 run.

8 Experiment 3: Varying Cooling Rate (α)

Using the 50-city instance with $\alpha \in \{0.90, 0.95, 0.995\}$:

Table 5: Experiment 3 Results

α	Best Cost	Temp Steps to $T_{\min}$	Time(s)
0.900	6104	132	0.0026
0.950	6117	270	0.0054
0.995	5940	2757	0.0200

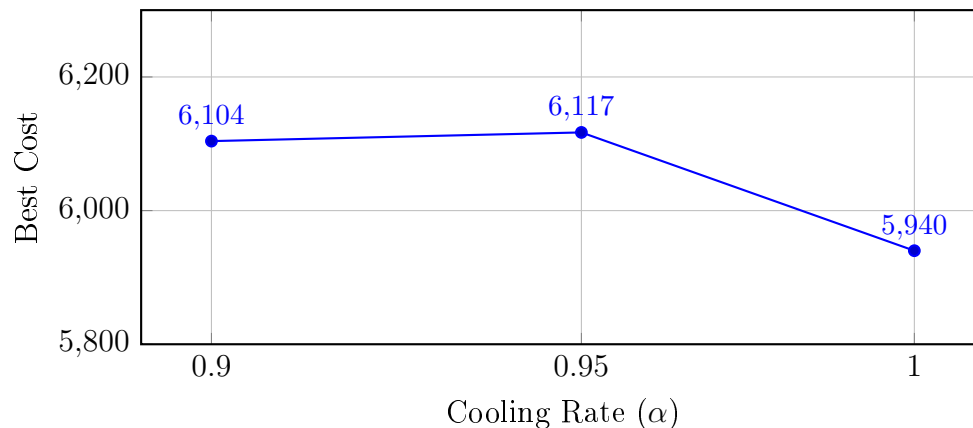


Figure 4: Best cost vs. cooling rate

Discussion:

- **Fast cooling** ($\alpha = 0.90$): Reaches $T_{\min}$ in only 132 temperature steps. The search spends very little time at each temperature and freezes quickly into a local optimum (cost = 6104). Computation is cheap (0.0026s).
- **Moderate cooling** ($\alpha = 0.95$): Takes 270 steps. Slightly worse than $\alpha = 0.90$ in this run (cost = 6117), likely due to random variation.
- **Slow cooling** ($\alpha = 0.995$): Takes 2,757 steps — far more temperature levels. This gives the algorithm much more time to explore the solution space and escape local optima, achieving the best cost of 5940. The computational cost (0.02s) is still very manageable.

9 Experiment 4: Random vs. Greedy Initialization

Using the 50-city instance, SA was run 5 times with each initialization method:

Table 6: Experiment 4 Results

Init Method	Best	Avg	Worst	Avg Time(s)
Random	5917	5974	6023	0.0196
Greedy	5981	6041	6207	0.0197

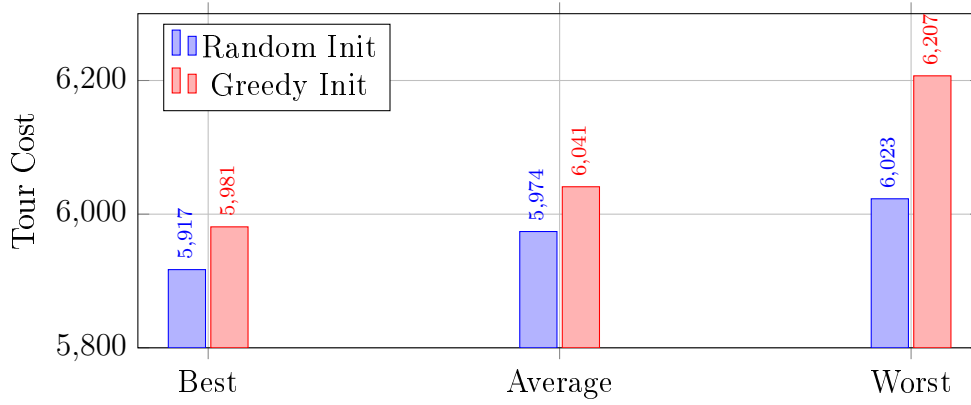


Figure 5: Random vs. Greedy initialization comparison

Discussion: In these runs, random initialization achieved better results than greedy initialization across all metrics (best, average, worst). This is because:

- Random initialization provides diverse starting points, giving SA a better chance of exploring different basins of attraction in the solution space.
- Greedy initialization starts SA in a nearby region of the search space. With a limited iteration budget, SA may not have enough iterations to move far from the greedy starting point.
- For larger iteration budgets, greedy initialization would likely outperform random initialization because it starts closer to a good solution.

This result highlights an important practical consideration: the effectiveness of the initialization strategy depends on the interaction between the starting solution quality and the available computational budget.

10 Complexity Analysis

10.1 Greedy Technique

The Nearest Neighbor algorithm has time complexity $O(N^2)$:

- At each of the N steps, the algorithm scans all N cities to find the nearest unvisited one.
- The tour cost calculation is $O(N)$.
- Overall: $O(N^2)$ for the tour construction, plus $O(N)$ for cost calculation.

Space complexity: $O(N^2)$ for the cost matrix.

10.2 Simulated Annealing

Let I be the total number of iterations:

- **Neighbor generation (2-opt reversal):** $O(N)$ per iteration.

- **Tour cost recalculation:** $O(N)$ per iteration (full recalculation).
- **Per-iteration cost:** $O(N)$.
- **Total time:** $O(I \cdot N)$.

Potential Optimization: Instead of recalculating the entire tour cost after each 2-opt reversal, we can compute only the delta in $O(1)$:

$$\Delta = \text{cost}(i-1, j) + \text{cost}(i, j+1) - \text{cost}(i-1, i) - \text{cost}(j, j+1)$$

This would reduce the per-iteration cost from $O(N)$ to $O(1)$, yielding an overall complexity of $O(I)$ for the inner loop. However, the current implementation uses full recalculation for simplicity and correctness.

Space complexity: $O(N^2)$ for the cost matrix.

11 Comparative Summary

Table 7: Algorithm Comparison

Aspect	Greedy	Simulated Annealing
Time Complexity	$O(N^2)$	$O(I \cdot N)$
Solution Quality	Local optimum	Near-global optimum
Determinism	Deterministic	Probabilistic
Escapes Local Optima	No	Yes
Parameters	None	$T_0, \alpha, T_{\min}$, iterations
Typical Improvement	—	3–20% over Greedy

12 Conclusion

This assignment demonstrates the trade-offs between a deterministic greedy heuristic and a probabilistic metaheuristic for the TSP:

1. The **Greedy algorithm** provides a fast $O(N^2)$ solution but gets trapped in local optima. It is useful as a baseline or as an initialization for more advanced methods.
2. **Simulated Annealing** consistently finds better tours (3–20% improvement) at the cost of more computation. Its ability to accept worse moves allows it to escape local optima.
3. **Parameter sensitivity** is critical: moderate initial temperature ($T_0 = 1000$) and slow cooling ($\alpha = 0.995$) yield the best results for the tested instances.
4. **Initialization strategy** interacts with the iteration budget: random initialization can outperform greedy initialization when the budget is limited, as it provides more diverse exploration.

13 Submission Contents

- `tsp2.cpp` — Complete C++ source code
- `report.tex` — This LaTeX report
- `report.txt` — Raw experimental output
- `Test1_10.txt` through `Test5_200.txt` — Generated test instances